

Assignment 6 – SQL QUERIES

Name: Aryan Ligade

Prn-124b1f155

class/div-Sy(A4)

Case Study: Employee Performance Appraisal System

Aim:

To create and implement different types of database triggers (BEFORE and AFTER INSERT, UPDATE, DELETE) on an Employee Management System to automate logging and validation

2. Problem Statement (Case Study)

Write all types of Database Triggers for suitable application.

3. Database Design

1. Employee Entity

This entity stores details of employees.

- emp_id → Integer, Primary Key
- name → String (cannot be NULL)
- salary → Integer
- rating → Integer

Each employee is uniquely identified by emp_id.

2. Appraisal_Log Entity

This entity stores logs generated automatically using triggers.

- log_id → Integer, Primary Key, Auto Increment
- emp_id → Foreign Key referencing Employee
- action → String (INSERT, UPDATE, DELETE)
- old_salary → Integer
- new_salary → Integer

- **log_date** → **Timestamp**

This table is used for auditing and tracking employee changes.

4. Employee_Department Entity

This table connects employees with departments.

- **emp_id** → Foreign Key referencing Employee
- **dept_id** → Foreign Key referencing Department

4. Database Schema

```
mysql> CREATE TABLE employees (  
->     emp_id INT PRIMARY KEY,  
->     name VARCHAR(50),  
->     salary INT,  
->     rating INT  
-> );  
Query OK, 0 rows affected (0.03 sec)
```

```
mysql>  
mysql> CREATE TABLE appraisal_log (  
->     log_id INT AUTO_INCREMENT PRIMARY KEY,  
->     emp_id INT,  
->     action VARCHAR(20),  
->     old_salary INT,  
->     new_salary INT,  
->     log_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
-> );  
Query OK, 0 rows affected (0.02 sec)
```

5. SQL Inputs and Outputs

1. INSERT Trigger (After Adding Employee)

```
mysql> CREATE TRIGGER trg_after_insert
-> AFTER INSERT ON employees
-> FOR EACH ROW
-> BEGIN
->     INSERT INTO appraisal_log(emp_id, action, new_salary)
->     VALUES (NEW.emp_id, 'INSERT', NEW.salary);
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO employees VALUES (101, 'Aryan', 50000, 4);
Query OK, 1 row affected (0.01 sec)
```

2. UPDATE Trigger (Appraisal Salary Change)

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_after_update
-> AFTER UPDATE ON employees
-> FOR EACH ROW
-> BEGIN
->     INSERT INTO appraisal_log(emp_id, action, old_salary, new_salary)
->     VALUES (NEW.emp_id, 'UPDATE', OLD.salary, NEW.salary);
-> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE employees
->
-> SET salary = 60000
->
-> WHERE emp_id = 101;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1 Changed: 1 Warnings: 0
```

3. DELETE Trigger (Employee Removal)

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_after_delete
  -> AFTER DELETE ON employees
  -> FOR EACH ROW
  -> BEGIN
  ->     INSERT INTO appraisal_log(emp_id, action, old_salary)
  ->     VALUES (OLD.emp_id, 'DELETE', OLD.salary);
  -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> INSERT INTO employees VALUES (101, 'Amit', 60000);
ERROR 1136 (21S01): Column count doesn't match value count at row 1
mysql> DELETE FROM employees
  ->
  -> WHERE emp_id = 101;
Query OK, 1 row affected (0.00 sec)
```

4. BONUS: BEFORE UPDATE Trigger (Auto Appraisal Based on Rating)

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_before_update
  -> BEFORE UPDATE ON employees
  -> FOR EACH ROW
  -> BEGIN
  ->     IF NEW.rating = 5 THEN
  ->         SET NEW.salary = NEW.salary + 10000;
  ->     ELSEIF NEW.rating = 4 THEN
  ->         SET NEW.salary = NEW.salary + 5000;
  ->     END IF;
  -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE employees
  ->
  -> SET salary = 60000, rating = 5
  ->
  -> WHERE emp_id = 102;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 0  Changed: 0  Warnings: 0
```

Ouput :

```
mysql> INSERT INTO employees VALUES (101, 'Amit', 50000, 4);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT * FROM appraisal_log;
+-----+-----+-----+-----+-----+-----+
| log_id | emp_id | action | old_salary | new_salary | log_date |
+-----+-----+-----+-----+-----+-----+
| 1 | 101 | INSERT | NULL | 50000 | 2026-04-16 03:00:17 |
| 2 | 101 | UPDATE | 50000 | 60000 | 2026-04-16 03:01:12 |
| 3 | 101 | DELETE | 60000 | NULL | 2026-04-16 03:10:18 |
| 4 | 101 | INSERT | NULL | 50000 | 2026-04-16 03:13:08 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)

mysql> UPDATE employees
-> SET salary = 60000
-> WHERE emp_id = 101;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1 Changed: 1 Warnings: 0

mysql> SELECT * FROM appraisal_log;
+-----+-----+-----+-----+-----+-----+
| log_id | emp_id | action | old_salary | new_salary | log_date |
+-----+-----+-----+-----+-----+-----+
| 1 | 101 | INSERT | NULL | 50000 | 2026-04-16 03:00:17 |
| 2 | 101 | UPDATE | 50000 | 60000 | 2026-04-16 03:01:12 |
| 3 | 101 | DELETE | 60000 | NULL | 2026-04-16 03:10:18 |
| 4 | 101 | INSERT | NULL | 50000 | 2026-04-16 03:13:08 |
| 5 | 101 | UPDATE | 50000 | 65000 | 2026-04-16 03:13:24 |
+-----+-----+-----+-----+-----+-----+
```

```
mysql> SELECT * FROM employees;
+-----+-----+-----+-----+
| emp_id | name | salary | rating |
+-----+-----+-----+-----+
| 101 | Amit | 65000 | 4 |
+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

7. Conclusion

In this experiment, we successfully designed and implemented various types of **database triggers** on an Employee Management System to automate important operations such as validation, auditing, and data consistency.

Triggers proved to be highly useful in ensuring that specific actions are executed automatically in response to database events like **INSERT, UPDATE, and DELETE**. By using **AFTER triggers**, we were able to maintain a complete log of employee activities in the appraisal_log table, which is essential for tracking changes and maintaining transparency. This demonstrates how triggers help in **auditing and historical record-keeping** without requiring manual intervention.

The **BEFORE UPDATE trigger** added an intelligent layer of business logic by automatically modifying employee salaries based on their performance rating. This highlights how triggers can enforce **business rules directly at the database level**, ensuring data integrity regardless of the application being used.

Additionally, the use of **OLD and NEW keywords** allowed us to access previous and updated values efficiently, making it possible to compare changes and store accurate logs.